WCHGUI 软件开发

参考手册

版本：1.2

https://wch.cn

1 概述

1.1 产品简介

WCHGUI 是 WCH（沁恒微电子）为旗下微控制器设计的轻量级图形用户界面库。该库专为嵌入式系统优化，提供直观的 API 接口，支持丰富的 UI 控件类型，搭配 WCHGUIDesigner.exe 使用使开发者能够快速构建美观的图形界面应用。

1.2 技术特性

- 轻量化设计：针对资源受限的嵌入式环境进行优化
- 模块化架构：清晰的分层设计便于扩展和维护
- 声明式 UI：通过配置文件定义界面元素，降低开发复杂度
- 多页面管理：支持页面间灵活切换
- 硬件抽象：统一的硬件接口便于移植
- 高效渲染：固定帧率渲染保证流畅体验

1.3 应用场景

- 智能家电控制面板
- 工业设备人机界面
- 医疗设备操作界面
- 物联网终端显示
- 教学演示平台

2 系统架构

2.1 核心文件

2.1.1 wui.h - 核心接口定义

定义了所有公共 API、数据结构和常量。

2.1.2 wui_page.c - 页面管理系统

管理多个 UI 页面及其生命周期。

2.1.3 wui_page.h - 页面管理接口

所有页面的枚举，以及页面初始化函数及页面循环函数的声明。

2.1.4 wui_page_x.c 页面 x 的具体实现

定义页面 UI 控件描述符。

2.1.5 images.h 图片资源管理接口

管理所有 UI 中使用的图片资源，提供统一的图片 ID 索引和元数据信息。

2.1.6 wui.asset.h 资源索引

定义字符串、字体、图片的枚举 ID。

2.1.7 wui_widget.h 控件系统

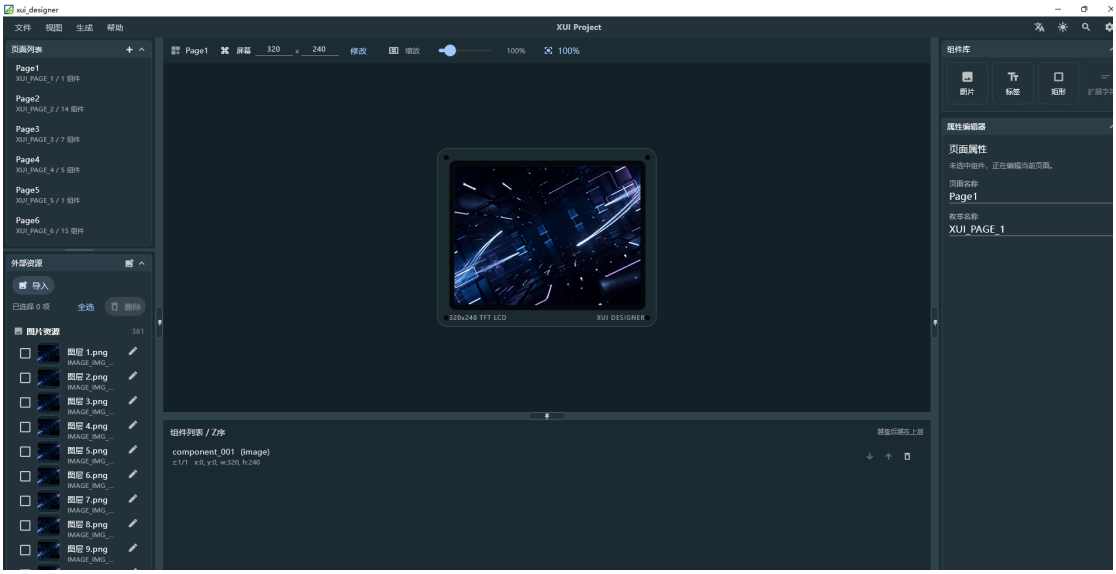
高级控件层，在 wui.h 的基础元素系统之上，封装了更复杂的交互组件。

3 快速入门指南

3.1 前期准备

3.1.1 UI 设计

通过开发的上位机完成对应的 UI 设计，上位机的具体操作步骤这里不在具体展开，我们只帮助开发者快速入门 WCHGUI。



在生成选项框中选择生成文件，此时会生成对应的 bin 文件和 c 语言文件。

images.bin	2026/8/3 18:02	BIN 文件	5,494 KB
images.h	2026/8/3 18:02	C Header 源文件	92 KB
manifest.txt	2026/8/3 18:02	文本文档	1 KB
wui_asset.c	2026/8/3 18:02	C 源文件	8 KB
wui_asset.h	2026/8/3 18:02	C Header 源文件	2 KB
wui_page.c	2026/8/3 18:02	C 源文件	2 KB
wui_page.h	2026/8/3 18:02	C Header 源文件	1 KB
wui_page_none.c	2026/8/3 10:29	C 源文件	1 KB
wui_page_none.h	2026/8/3 18:02	C Header 源文件	1 KB
wui_page_page_1.c	2026/8/3 10:29	C 源文件	1 KB
wui_page_page_1.h	2026/8/3 18:02	C Header 源文件	6 KB

3.1.2 工程文件建立

在已经完成 LCD 屏幕驱动和外部 Flash 驱动的工程项目中添加以下目录（用户可以自行调整），注：wui 文件夹下的文件不必替换，用户若是从 0 开始新建项目则需手动添加 WCH 提供的 wui.c 以及 wui_widget.h 两个文件：

```
Project/
└── LCD_wui/
    ├── User/
    |   ├── wui/       # xui核心库
    |   ├── wui_port/  # xui硬件接口定义处
    |   └── wui_user/  # 用户自定义页面
```

3.2 工程初始化

3.2.1 头文件引用

```
#include "wui.h"
#include "wui_port.h"
#include "wui_user/wui_page.h"
```

3.2.2 主函数框架梳理

为帮助开发者快速理解 WCHGUI 框架的逻辑，此处对不必要的功能进行裁剪，开发者基本只需要关注自己的业务层即可，无需多余的操作，仅在初始化阶段对 WCHGUI 进行界面初始化和加载初始界面即可，开发者需自定义 UI 帧率控制的时基计时器，用于精确控制图形界面的刷新频率。

```
#include "wui.h"
#include "wui_port.h"
#include "wui_user/wui_page.h"

int main(void)
{
  // 系统基础初始化
 SystemCoreClockUpdate();
 Delay_Init();
 USART_Printf_Init(115200);

 // 硬件初始化
 LCD_Init();      // LCD 初始化
 SPI_FLASH_Init();    // Flash 存储初始化

 // wui 系统初始化
 wui_init_data_t data = {
  .page_count = WUI_PAGE_COUNT,
  .height = get_user_lcd_height, //用户屏幕的高度
  .width = get_user_lcd_width,  //用户屏幕的宽度
  .heap_addr = wui_heap_buffer, //wui 堆内存起始地址
  .heap_size = sizeof(wui_heap_buffer),//wui 堆内存大小

 };
```

```
 wui_init(&data);          // 初始化页面系统
// 注册用户实现的 HAL 接口
wui_register_hal(wui_get_hal());
 wui_page_switch(WUI_START_PAGE);    // 初始加载界面

 // 主循环
      while(1) {
    uint16_t now = TIM4->CNT;
    if (now > 16)
    {
     now     = now / 8; // 缩放时间片
     TIM4->CNT = 0;          // 清零定时器
     wui_page_ui_tick(now);// 向 UI 系统传递经过的时间片
    }
  // 更新 UI 系统
    wui_page_ui_update();

  // 处理用户其他逻辑
  }
}
```

3.3 绘制页面介绍

为了方便开发者快速了解上位机生成的 c 文件和 h 文件中的内容，此处创建一个只含有一个背景图片的显示页面：

3.3.1 实现页面生命周期函数

```
// 按键处理函数
static void page_1_on_key(uint32_t key)
{
   (void)key; // 用户自定义
}
// 页面初始化函数
void wui_ page_1_init(void)
{
 // 初始化页面控件
 wui_page_init(widgets, sizeof(widgets) / sizeof(widgets[0]));
 // 注册按键处理函数
 wui_register_key_handler(page_1_on_key);
}

// 页面循环函数
void wui_page_page_1_loop(void)
{
 // 页面特定的循环逻辑（可选）
}
```

3.3.2 注册页面

在 wui_page.c 中注册新页面：

```
const wui_page_desc_t wui_pages[WUI_PAGE_COUNT] = {
 // ... 其他页面 ...
 [WUI_PAGE_1] = {
  .page_name = "Page1",
  .page_init = wui_page_page1_init,
  .page_loop = wui_page_page1_loop,
  .user_data = NULL,
 },
 // ... 其他页面 ...
};
```

上述即为核心逻辑，用户只需在相应的代码区填入自己的交互逻辑即可实现页面切换、更新图片及文字等。

4 API 参考手册

4.1 wui 系统初始化 API

4.1.1 wui_init(wui_init_data_t *data)

```
void wui_init(wui_init_data_t *data);
```

功能描述	初始化 wui 系统，设置页面管理器参数
输入参数	Data：包含屏幕参数及页面数量的结构体
输出参数	无
返回值	无

使用示例：

```
// 初始化 wui 系统，data 中包含屏幕的长、宽及页面的总数量
wui_init(&data);
```

注意事项：

1. 必须在调用其他 wui 函数前调用此函数；
2. page_count 必须大于 0 且不超过最大页面限制。

4.1.2 wui_register_hal(const wui_hal_t* hal)

```
void wui_register_hal(const wui_hal_t* hal);
```

功能描述	注册 WUI 硬件抽象层接口
输入参数	wui_hal_t* 类型的实例指针
输出参数	无
返回值	无。

4.1.3 const char* wui_get_version(void)

```
const char* wui_get_version(void);
```

功能描述	获取 WUI 框架版本字符串。
输入参数	无
输出参数	无
返回值	版本号字符串。

4.2 页面管理 API

4.2.1 wui_page_init(const wui_widget_desc_t* widgets, uint8_t widget_count)

```
void wui_page_init(const wui_widget_desc_t* widgets, uint8_t widget_count);
```

功能描述	初始化当前页面的控件系统
输入参数	widgets ：控件描述符数组
输入参数	count ：控件数量
输出参数	无
返回值	无

使用示例：

```
wui_page_init(widgets, sizeof(widgets) / sizeof(widgets[0]));
```

4.2.2 void wui_page_ui_tick (uint32_t tick)

```
uint32_t wui_page_ui_tick (uint32_t tick);
```

功能描述	记录时间的流逝，向 UI 系统传递经过的时间片
输入参数	tick：经过的时间单位数量
输出参数	无
返回值	无

4.2.3 uint32_t wui_page_ui_update (void)

```
void wui_page_ui_update (void);
```

功能描述	更新 UI 系统状态，处理动画等
输入参数	无
输出参数	无
返回值	返回是否有需要更新发生： 0：无更新发生， 非 0：有更新发生。

4.2.4 void wui_page_switch(uint16_t page)

```
void wui_page_switch(uint16_t page);
```

功能描述	切换到指定页面
输入参数	Page：目标页面索引（从 0 开始）
输出参数	无
返回值	无

使用示例：

```
wui_page_switch(WUI_PAGE_2); // 切换到第 2 页
```

错误处理：

如果 page 参数超出有效范围，函数将忽略该请求

4.2.5 const wui_page_desc_t* wui_page_get_desc(uint16_t page)

```
const wui_page_desc_t* wui_page_get_desc(uint16_t page);
```

功能描述	获取指定页面的描述信息
输入参数	Page：页面索引
输出参数	无
返回值	成功：指向页面描述结构的指针； 失败：NULL 指针。

使用示例：

```
const wui_page_desc_t* desc = wui_page_get_desc(WUI_PAGE_1);
if(desc != NULL) {
 printf("Page name: %s\n", desc->page_name);
}
```

4.2.6 const char* wui_page_get_name(uint16_t page)

```
const char* wui_page_get_name(uint16_t page);
```

功能描述	获取指定页面的名称
输入参数	Page：页面索引
输出参数	无
返回值	成功：页面名称字符串； 失败：NULL 指针。

使用示例：

```
 const char* page_name = wui_page_get_name(WUI_PAGE_1);
if(page_name != NULL) {
 printf("Current page: %s\n", page_name);
}
```

4.3 输入处理 API

4.3.1 void wui_register_key_handler(void (*handler)(uint32_t key))

```
void wui_register_key_handler(void (*handler)(uint32_t key));
```

功能描述	注册按键处理回调函数
输入参数	handler：按键处理函数指针
输出参数	无
返回值	无

使用示例：

```
 void page_1_on_key(uint32_t key) {
 switch(key) {
  case 'LEFT': printf("Left pressed\n"); break;
  case 'RIGHT': printf("Right pressed\n"); break;
  case 'ENTER': printf("Enter pressed\n"); break;
 }
```

```
}

// 注册按键处理器
wui_register_key_handler(page_1_on_key);
```

4.3.2 void wui_set_key_event(uint32_t key)

```
void wui_set_key_event(uint32_t key);
```

功能描述	触发按键事件
输入参数	Key：按键值
输出参数	无
返回值	无

使用示例：

```
wui_set_key_event( 'ENTER' ); // 模拟按下确认键
```

4.4 进度条控件 API

4.4.1 wui_widget_pb_set_value(uint16_t widget_id, uint16_t value)

```
Void wui_widget_pb_set_value(uint16_t widget_id, uint16_t value);
```

功能描述	设置进度条当前值
输入参数	widget_id ：UI 控件 ID
输入参数	value ：需要设置的值
输出参数	无
返回值	无

使用示例：

```
wui_widget_pb_set_value(COMPONENT_001, 20);
```

4.4.2 wui_widget_pb_get_value(uint16_t widget_id)

```
uint8_t wui_widget_pb_get_value(uint16_t widget_id);
```

功能描述	获取进度条的当前值
输入参数	widget_id ：UI 控件 ID
输出参数	无
返回值	进度条的值 （范围 0-100）

使用示例：

```
uint8_t pbvalue = wui_widget_pb_get_value (COMPONENT_001);
```

4.5 动画控件 API

4.5.1 wui_widget_anime_play(uint16_t widget_id)

```
Void wui_widget_anime_play(uint16_t widget_id);
```

功能描述	播放动画
输入参数	widget_id ：UI 控件 ID
输出参数	无

返回值	无

使用示例：

```
wui_widget_anime_play(COMPONENT_001);
```

4.5.2 wui_widget_anime_pause(uint16_t widget_id)

```
Void wui_widget_anime_pause(uint16_t widget_id);
```

功能描述	暂停动画
输入参数	widget_id ：UI 控件 ID
输出参数	无
返回值	无

使用示例：

```
wui_widget_anime_pause (COMPONENT_001);
```

4.5.3 wui_widget_anime_reset(uint16_t widget_id)

```
Void wui_widget_anime_reset(uint16_t widget_id);
```

功能描述	重置动画到第一帧
输入参数	widget_id ：UI 控件 ID
输出参数	无
返回值	无

使用示例：

```
wui_widget_anime_reset(COMPONENT_001);
```

4.5.4 wui_widget_anime_is_completed (uint16_t widget_id)

```
uint8_t wui_widget_anime_is_completed(uint16_t widget_id);
```

功能描述	查询动画是否播放完成
输入参数	widget_id ：UI 控件 ID
输出参数	无
返回值	0=未完成，1=已完成

使用示例：

```
uint8_t iscomplete = wui_widget_anime_is_completed(COMPONENT_001);
```

4.6 开关控件 API

4.6.1 wui_widget_switch_toggle(uint16_t widget_id)

```
void wui_widget_switch_toggle(uint16_t widget_id);
```

功能描述	翻转开关状态
输入参数	widget_id ：UI 控件 ID
输出参数	无
返回值	无

使用示例：

```
wui_widget_switch_toggle (COMPONENT_001);
```

4.6.2 wui_widget_switch_set_on(uint16_t widget_id, uint8_t on)

```
Void wui_widget_switch_set_on(uint16_t widget_id, uint8_t on);
```

功能描述	设置开关状态
输入参数	widget_id ：UI 控件 ID
输入参数	on： 0=关，1=开
输出参数	无
返回值	无

使用示例：

```
wui_widget_switch_set_on(COMPONENT_001, 1);//开关控件开关状态设置为开
```

4.6.3 wui_widget_switch_is_on(uint16_t widget_id)

```
uint8_t wui_widget_switch_is_on(uint16_t widget_id);
```

功能描述	查询开关当前状态
输入参数	id：UI 控件 ID
输出参数	无
返回值	0：关闭状态 1：开启状态

使用示例：

```
uint8_t key_state = wui_widget_switch_is_on(COMPONENT_001); //判断开关控件的状态
```

4.7 矩形控件 API

4.7.1 wui_widget_rect_set_color(uint16_t widget_id, uint16_t color)

```
Void wui_widget_rect_set_color(uint16_t widget_id, uint16_t color);
```

功能描述	设置矩形颜色
输入参数	widget_id ：UI 控件 ID
输入参数	color：矩形颜色
输出参数	无
返回值	无

使用示例：

```
wui_widget_rect_set_color (COMPONENT_001,0xF800); // 设置矩形控件为红色
```

4.7.2 wui_widget_rect_get_color (uint16_t widget_id)

```
uint16_t wui_widget_rect_get_color(uint16_t widget_id);
```

功能描述	获取矩形颜色，大端值
输入参数	widget_id ：UI 控件 ID
输出参数	无
返回值	矩形控件颜色值

使用示例：

```
color_t = wui_widget_rect_get_color (COMPONENT_001);// 获取矩形控件颜色值
```

4.8 按钮控件 API

4.8.1 wui_widget_button_press (uint16_t widget_id)

```
void wui_widget_button_press(uint16_t widget_id);
```

功能描述	模拟按钮按下
输入参数	widget_id ：UI 控件 ID
输出参数	无
返回值	无

使用示例：

```
wui_widget_button_press (COMPONENT_001);
```

4.8.2 wui_widget_button_release (uint16_t widget_id)

```
Void wui_widget_button_release (uint16_t widget_id);
```

功能描述	模拟按钮释放
输入参数	widget_id ：UI 控件 ID
输出参数	无
返回值	无

使用示例：

```
wui_widget_button_release(COMPONENT_001);
```

4.8.3 wui_widget_button_is_pressed(uint16_t widget_id)

```
uint8_t wui_widget_button_is_pressed(uint16_t widget_id);
```

功能描述	查询按钮是否处于按下状态
输入参数	id：UI 控件 ID
输出参数	无
返回值	0：按下状态 1：释放状态

使用示例：

```
uint8_t button_state = wui_widget_button_is_pressed(COMPONENT_001); //判断按钮控件的状态
```

4.9 标签控件 API

4.9.1 wui_widget_simplestring_set_str(uint16_t widget_id, const char* str)

```
void wui_widget_simplestring_set_str(uint16_t widget_id, const char* str);
```

功能描述	设置标签文本内容
输入参数	widget_id ：UI 控件 ID
输入参数	str ：字符串指针
输出参数	无
返回值	无

使用示例：

```
const char status_text[] = "Ready";
wui_widget_simplestring_set_str (COMPONENT_001,status_text);
```

4.9.2 wui_widget_simplestring_set_align(uint16_t widget_id, uint8_t align)

```
Void wui_widget_simplestring_set_align(uint16_t widget_id, uint8_t align);
```

功能描述	设置标签文本对齐方式
输入参数	widget_id ：UI 控件 ID
输入参数	align ：文本对齐方式
输出参数	无
返回值	无

使用示例：

```
wui_widget_simplestring_set_align(COMPONENT_001,WUI_SIMPLESTRING_ALIGN_LEFT); // 设置文本对比方式为左对齐
```

4.9.3 wui_widget_simplestring_set_color(uint16_t widget_id, uint16_t fg_color, uint16_t bg_color);

```
Void wui_widget_simplestring_set_color(uint16_t widget_id, uint16_t fg_color, uint16_t bg_color);
```

功能描述	标签文本颜色设置
输入参数	widget_id ：UI 控件 ID
输入参数	fg_color ：前景色 RGB565 格式
输入参数	bg_color ：背景色 RGB565 格式
输出参数	无
返回值	无

使用示例：

```
wui_widget_simplestring_set_color(COMPONENT_002,RGB24_TO_RGB565(0xEF5350),0x0000);
```

4.9.4 wui_widget_simplestring_get_str (uint16_t widget_id)

```
const char* wui_widget_simplestring_get_str(uint16_t widget_id);
```

功能描述	获取当前标签文本
输入参数	id：UI 控件 ID
输出参数	无
返回值	当前文本内容

使用示例：

```
str_value_t  = wui_widget_simplestring_get_str (COMPONENT_001);
```

4.9.5 wui_widget_simplestring_get_align(uint16_t widget_id)

```
uint8_t wui_widget_simplestring_get_align(uint16_t widget_id);
```

功能描述	获取标签文本对齐方式
输入参数	id：UI 控件 ID
输出参数	无
返回值	标签文本对齐方式

使用示例：

```
uint8_t lable_ align = wui_widget_simplestring_get_align (COMPONENT_001);
```

4.10 艺术字控件 API

4.10.1 wui_widget_exstring_set_str (uint16_t widget_id, const char* str)

```
void wui_widget_exstring_set_str(uint16_t widget_id, const char* str);
```

功能描述	设置艺术字文本内容
输入参数	widget_id ：UI 控件 ID
输入参数	str ：字符串指针
输出参数	无
返回值	无

使用示例：

```
const char status_text[] = "Ready";
wui_widget_exstring_set_str(COMPONENT_001,status_text);
```

4.10.2 wui_widget_exstring_set_align (uint16_t widget_id, uint8_t align)

```
Void wui_widget_simplestring_set_align (uint16_t widget_id, uint8_t align);
```

功能描述	设置艺术字文本对齐方式
输入参数	widget_id ：UI 控件 ID
输入参数	align ：文本对齐方式
输出参数	无
返回值	无

使用示例：

```
wui_widget_simplestring_set_align(COMPONENT_001, WUI_EXSTRING_ALIGN_LEFT); // 设置艺术字的文本对比方式为左对齐
```

4.10.3 wui_widget_exstring_get_str(uint16_t widget_id)

```
const char* wui_widget_exstring_get_str(uint16_t widget_id);
```

功能描述	获取当前艺术字文本
输入参数	id：UI 控件 ID
输出参数	无
返回值	当前文本内容

使用示例：

```
str_value_t  = wui_widget_exstring_get_str(COMPONENT_001);
```

4.10.4 wui_widget_exstring_get_align (uint16_t widget_id)

```
uint8_t wui_widget_exstring_get_align(uint16_t widget_id);
```

功能描述	获取艺术字文本对齐方式
输入参数	id：UI 控件 ID
输出参数	无
返回值	艺术字文本对齐方式

使用示例：

```
uint8_t lable_ align = wui_widget_exstring_get_align(COMPONENT_001);
```

4.11 图片控件 API

4.11.1 wui_widget_staticimg_set_img_id(uint16_t widget_id, uint16_t img_id)

```
Void wui_widget_staticimg_set_img_id(uint16_t widget_id, uint16_t img_id);
```

功能描述	设置图片控件中的图片
输入参数	widget_id ：UI 控件 ID
输入参数	img_id ：图片 ID
输出参数	无
返回值	无

使用示例：

```
wui_widget_staticimg_set_img_id(COMPONENT_001,IMAGE_IMG_IMAGE); // 设置图片控件图片为 IMAGE_IMG_IMAGE
```

4.11.2 wui_widget_staticimg_get_img_id (uint16_t widget_id)

```
uint16_t wui_widget_staticimg_get_img_id (uint16_t widget_id);
```

功能描述	获取图片 ID
输入参数	widget_id ：UI 控件 ID
输出参数	无
返回值	图片 ID 值

使用示例：

```
Image_id_t = wui_widget_rect_get_color (COMPONENT_001);// 获取图片控件中的图片 ID
```

4.12 静态文字控件 API

请注意静态文字控件是将字符串转化为图片的形式，因此与图片控件 API 实际是一致的。

4.13 图片滑块控件 API

4.13.1 wui_widget_imgslider_set_value(uint16_t widget_id, uint8_t current_value);

```
Void wui_widget_imgslider_set_value(uint16_t widget_id, uint8_t current_value);
```

功能描述	设置滑块当前值
输入参数	widget_id ：UI 控件 ID
输入参数	current_value ：需要设置的值
输出参数	无
返回值	无

使用示例：

```
wui_widget_imgslider_set_value (COMPONENT_001, 20);
```

4.13.2 wui_widget_imgslider_get_value (uint16_t widget_id)

```
uint8_t wui_widget_imgslider_get_value (uint16_t widget_id);
```

功能描述	获取滑块的当前值
输入参数	widget_id ：UI 控件 ID
输出参数	无
返回值	进度条的值 （范围 0-100）

使用示例：

```
uint8_t value = wui_widget_imgslider_get_value (COMPONENT_001);
```

5 数据结构定义

5.1 硬件抽象层结构体

```
typedef struct
{
   wui_touch_read_cb_t     touch_read;     //触摸屏读取回调函数
   wui_flash_read_cb_t     flash_read;     //数据读取回调函数
   wui_lcd_set_windows_cb_t lcd_set_windows; //设置显示窗口回调函数
   wui_lcd_dma_start_cb_t   lcd_dma_start;   // DMA 传输开始回调
   wui_lcd_dma_send_cb_t    lcd_dma_send;    // DMA 数据发送回调
   wui_lcd_dma_wait_cb_t    lcd_dma_wait;    // DMA 等待传输完成回调
   wui_lcd_dma_end_cb_t     lcd_dma_end;     // DMA 传输结束回调
} wui_hal_t;
```

5.2 字体描述符集合

```
typedef struct {
   uint16_t height;      // 字符高度
   uint16_t width;       // 字符宽度
   uint8_t  cnt;         // 字符集中的字符数量
   uint8_t  start_char;   // 起始字符 ASCII 码
   uint16_t first_img_id; // 首字符对应的图片 ID
} wui_font_t;
```

5.3 页面描述符

```
typedef struct
{
 uint16_t   bg_color;   //背景颜色
 const char* page_name;  // 页面名称
 void (*page_init)(void); // 页面初始化函数
 void (*page_loop)(void); // 页面循环函数
 void* user_data;      // 用户数据指针
} wui_page_desc_t;
```

5.4 初始化信息

```
typedef struct {
   uint16_t width;       // 屏幕宽度
```

```
    uint16_t height;      // 屏幕高度
    uint16_t page_count;   // 页面总数
    uint8_t* heap_addr;    // 堆内存起始地址
    size_t   heap_size;    // 堆内存大小
} wui_init_data_t;
```

5.5 控件类型枚举

```
typedef enum {
    Wui_w_None,           //NONE
    Wui_w_Anime,         //  动画
    Wui_w_Switch,         //  开关
    Wui_w_Res0,          //  保留 0
    Wui_w_MultiState,     //  多态图片
    Wui_w_StaticImage,    //  静态图片
    Wui_w_QRCode,         //  二维码
    Wui_w_Rect,           //  矩形
    Wui_w_Res1,          //  保留 1
    Wui_w_Res2,          //  保留 2
    Wui_w_Res3,          //  保留 3
    Wui_w_ProgressBar,    //  进度条
    Wui_w_Button,         //  按钮
    Wui_w_ImgSlider,        //图片滑块
    Wui_w_SimpleString,    //  标签文本
    Wui_w_ExString,       //  艺术字
    Wui_w_COUNT           //  总数
    } wui_widget_type_t;
```

5.6 控件属性结构体

5.6.1 进度条

```
typedef struct {
  uint8_t  direction;      //方向
    uint16_t min_value;     // 最小值
    uint16_t max_value;     // 最大值
    uint16_t current_value; // 当前值
    uint16_t bg_color;      // 背景色
    uint16_t fill_color;     // 填充颜色
    uint16_t thumb_color;   // 滑块颜色
     uint16_t thumb_w;      //滑块宽度
} wui_widget_pb_props_t;
```

5.6.2 动画

```
typedef struct {
    uint16_t start_img_id; // 首帧图片 ID
    uint16_t frame_count;   // 总帧数
    uint8_t  fps;          // 播放帧率（帧/秒）
    uint8_t  loop;         // 0=单次，1=循环
```

```
} wui_widget_anime_props_t;
```

5.6.3 开关

```
typedef struct {
    uint16_t on_img_id;   // "开"状态图片
    uint16_t off_img_id;  // "关"状态图片
    uint8_t  is_on;       // 当前状态
} wui_widget_switch_props_t;
```

5.6.4 二维码

```
typedef struct {
    uint16_t img_id; // 二维码图片 ID
} wui_widget_qrcode_props_t;
```

5.6.5 矩形

```
typedef struct {
    uint16_t color; // RGB565 颜色
} wui_widget_rect_props_t;
```

5.6.6 按钮

```
typedef struct {
    uint16_t normal_img_id;   // 释放状态
    uint16_t pressed_img_id;  // 按下状态
    uint8_t  is_pressed;      // 当前是否按下
} wui_widget_button_props_t;
```

5.6.7 标签文本

```
#define WUI_SIMPLESTRING_ALIGN_LEFT   0 // 左对齐
#define WUI_SIMPLESTRING_ALIGN_RIGHT  1 // 右对齐
#define WUI_SIMPLESTRING_ALIGN_CENTER 2 // 居中对齐

typedef struct {
    wui_str_id_t str_id;    // 字符串资源 ID
    uint16_t     fg_color;  // 前景色
    uint16_t     bg_color;  // 背景色
    uint8_t      align;     // 对齐方式
    wui_bitmap_font_id_t font_id; // 字体 ID
} wui_widget_simplestring_props_t;
```

5.6.8 艺术字

```
#define WUI_EXSTRING_ALIGN_LEFT   0 // 左对齐
#define WUI_EXSTRING_ALIGN_RIGHT  1 // 右对齐
#define WUI_EXSTRING_ALIGN_CENTER 2 // 居中对齐
```

```
typedef struct {
   uint8_t type;  // 文本类型（预留）
   uint8_t align; // 对齐方式
   union {
      struct {
         uint16_t fg_color;
         uint16_t bg_color;
      };
      void* res; // 预留扩展指针
   };
   wui_str_id_t  str_id;  // 标签字符串 ID
   wui_font_id_t font_id; // 字体 ID
} wui_widget_exstring_props_t;
```

5.6.9 图片

```
typedef struct
{
   uint16_t img_id; // 图片 ID
} wui_widget_staticimg_props_t;

typedef struct
{
   wui_imgtable_id_t imgtable_id;   // 图片 ID
   uint8_t        state_count;   // 图片序列总数
   uint8_t        default_state; // 默认的图片序列
} wui_widget_multistate_props_t;
```

5.6.10 图片滑块

```
typedef struct
{
   uint16_t full_img_id;     //填充部分的图片 ID
   uint16_t empty_img_id;     //空白部分的图片 ID
   uint8_t  current_value; //当前进度值
   uint8_t  direction;     //滑块方向
} wui_widget_imgslider_props_t;
```

5.7 控件描述符

```
typedef struct {
   uint16_t       widget_id; // 控件 ID（页面内唯一）
   wui_widget_type_t type;      // 控件类型
   uint16_t       x, y, w, h; // 位置和尺寸
   uint8_t        flags;     // 标志位
   union {
      wui_widget_anime_props_t      anime;
```

```
    wui_widget_switch_props_t      switch_;
    wui_widget_multistate_props_t   multistate;
    wui_widget_staticimg_props_t    staticimg;
    wui_widget_qrcode_props_t      qrcode;
    wui_widget_rect_props_t        rect;
    wui_widget_pb_props_t         progress;
    wui_widget_button_props_t      button;
    wui_widget_imgslider_props_t    imgslider;
    wui_widget_simplestring_props_t simplestring;
     wui_widget_exstring_props_t    exstring;
  } props; // 类型对应的属性
} wui_widget_desc_t;

/* Bitmap font structure */
typedef struct
{
  uint8_t      width;      /* Font width */
  uint8_t      height;     /* Font height */
  uint8_t      start_char;  /* Start character */
  uint8_t      cnt;        /* Character count */
  const uint8_t* data;       /* Font data pointer */
} wui_bitfont_t;
```

5.8 触摸事件类型枚举

```
typedef enum
{
  WUI_EVENT_NONE = 0, //无事件

  WUI_EVENT_TOUCH_DOWN      = 1, //触摸按下
  WUI_EVENT_TOUCH_MOVE      = 2, //触摸滑动
  WUI_EVENT_TOUCH_LONGPRESS  = 3, //触摸长按
  WUI_EVENT_TOUCH_UP        = 4, //触摸抬起

  WUI_EVENT_WIDGET_CLICK      = 0x10, //单击事件
  WUI_EVENT_WIDGET_LONG_PRESS  = 0x11, //长按事件
  WUI_EVENT_WIDGET_CHANGE     = 0x20, //数值改变事件，例如进度条

} _wui_event_t;
```

5.9 触摸区域

```
typedef struct
{
  uint16_t widget_id; //控件 ID（页面内唯一）
  uint16_t x, y, w, h; //触摸区域
} wui_touch_area_t;
```